Comms Data Interchange Format

Specification Version 1.1

IQGeo, NINE Hills Road, Cambridge, CB2 IGE, United Kingdom.
Telephone: +44 (0) 1223 606655. Website: https://www.iqgeo.com/.

Version History

CDIF Version	Product Version	Comment
1.0	2.1	Initial version
1.1	3.0	Add support for line of count, cable offset geometry and cable segment association to equipment.

Contents

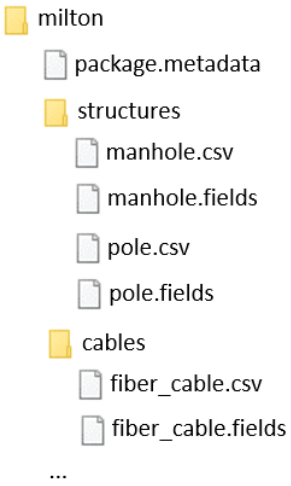
1 Overview 4
1.1 Package Structure 4
1.2 File Format 4
1.3 Feature Categories 5
1.4 Feature Identifiers 5
2 Feature Data 6
2.1 Structure 6
2.2 Route 6
2.3 Equipment 6
2.4 Conduit 7
2.5 Tube 7
2.6 Tube Segment 7
2.7 Cable 8
2.8 Cable Slack 8
2.9 Cable Segment 9
2.10 Connection 9
2.11 Line of Count 10
2.12 Line of Count Section 10
3 Field MetaData 12
4 Package Metadata 13

1 Overview

CDIF is a IQGeo format for exchanging telecom network data between database systems, retaining connectivity and containment relationships.

1.1 Package Structure

A CDIF package consists of a set of comma-separated variable (CSV) files. It is usually supplied in zipped format. A typical structure is shown below.



Each package file contains one of the following types of data:

File Type	Suffix	Description	Optional
Feature data	.csv	ID, location and attributes of instances of a feature type	No
Field metadata	.fields	Describes attributes of a feature type (data type, units)	Yes
Package metadata	.metadata	Global package properties (version, coordinate system, ...)	Yes

By convention, feature and field metadata files are grouped into sub-directories based on the *category* to which they belong. However, any file grouping (or none) is permitted.

1.2 File Format

All files have the following properties:

- Comma-separated variable text (CSV) with headings row
- Character encoding UTF8
- Geometry values represented as EWKB (extended well-known binary format)
- Boolean values represented as *true* or *false*
- Date values represented as *%Y-%m-%d*
- Timestamp files represented as *%Y-%m-%dT%H:%M:%S.%f*
- Null value indicated by empty string

For feature and field metadata files the first component of the file name defines the feature type to which the file relates e.g. *pole.1.csv* contains features of type *pole*.

The coordinate system in which geometries are represented can be specified in the package metadata. If not provided it is assumed to be WGS84 long/lat decimal degrees. Note that a feature can have more than one geometry field.

1.3 Feature Categories

The attributes provided for a feature type depend on the *category* to which it belongs. Categories are as follows:

Category	Description
Structure	A building, manhole, pole or other top-level point feature.
Route	A path between two structures (a trench, aerial route or similar).
Equipment	A signal carrying device (splitter, mux, ...) or enclosure (splice closure, rack, room, ...) within a structure
Conduit	A simple duct running between two structures
Tube	A duct that can potentially pass unbroken through structures
Tube Segment	A section of a tube between two structures
Cable	A pyhsical cable within the network
Cable Slack	A coil of cable within a structure
Cable Segment	A section of a cable between two structures (or within a structure)
Connection	A fiber-level connection from one range of fibers (or ports) to another range of fibers (or ports)
Line of Count	Assignment of name and status to strands in the network.

1.4 Feature Identifiers

Each feature record has a ID that is globally unique within the package. This allows it to be referenced unambiguously in connectivity and containment relationships.

By convention, feature identifiers usually have the form:

<feature_type>/<row_no> e.g. **pole/262**

However, any string or number can be used provided it is unique within the package.

2 Feature Data

This section describes the table structure for each type of feature data that can occur in the package.

2.1 Structure

A *structure* models a building, manhole, pole or other top-level point feature.

Field	Type	Optional	Comments
id	feature_id	No	Unique ID
location	point	No	Location in geographic world
myw_orientation_location	float	Yes	Symbol orientation (degrees clockwise from north)
<feature-specific field>	...	Yes	Feature type-specific attribute
...	...	...	...

Notes:

- *Feature-specific fields* will vary by structure type (height of pole, specification of manhole, ...)
- The package will contain a separate file for each structure type present in the data.

2.2 Route

A *route* models an direct path between two structures (a trench, aerial route or similar).

Field	Type	Optional	Comments
id	feature_id	No	Unique ID
in_structure	feature_id	No	Start structure (if any)
out_structure	feature_id	No	End structure (if any)
path	linestring	No	Path in geographic world (from start to end)
length	double	Yes	Measured length (in m)
<feature-specific field>	...	Yes	Feature type-specific attribute
...	...	...	

Notes:

- *Feature-specific fields* will vary by route type (depth of trench, owner of strand, ...)
- The package will contain a separate file for each route type present in the data.

2.3 Equipment

An *equipment* feature models a signal carrying device (splitter, mux, ...) or enclosure (splice closure, rack, room, ...).

Field	Type	Optional	Comments
id	feature_id	No	Unique ID
n_<tech>_in_ports	integer	Yes	Number of fiber input ports (if any). <tech> is one of fiber, copper or coax
n_<tech>_out_ports	integer	Yes	Number of fiber output ports (if any). <tech> is one of fiber, copper or coax.
housing	feature_id	No	Equipment or structure that is direct container

root_housing	feature_id	Yes	Structure in which self is housed
location	point	Yes	Location in geographic world
offset_geom	Point	Yes	Offset Location (for coax equipment)
<feature-specific field>	...	Yes	Feature type-specific attribute
...	...		...

Notes:

- If *root_housing* and *location* are supplied their values must match those of *housing*
- *Feature-specific fields* vary by equipment type type (asset ID, room number, ...)
- The package will contain a separate file for each equipment type present in the data.

2.4 Conduit

A *conduit* models models an duct running between two structures. Conduits can be contained directly within a route or nested within other conduits.

Field	Type	Optional	Comments
id	feature_id	No	Unique ID
housing	feature_id	No	Conduit or route that is direct container
root_housing	feature_id	Yes	Route in which rus
in_structure	feature_id	Yes	Start structure
out_structure	feature_id	Yes	End structure
path	linestring	Yes	Path in geographic world from start to end
<feature-specific field>	...	Yes	Feature type-specific attribute
...	...	...	...

Notes:

- If *root_housing, in_structure, out_structure* and *path* are supplied their values must match those of *housing*
- *Feature-specific fields* vary by conduit type
- The package will contain a separate file for each conduit type present in the data.

2.5 Tube

A *tube* models a duct that can potentially pass unbroken through structures e.g. a blown fiber tube. Internally it consists of a chain of *tube segments*.

Field	Type	Optional	Comments
id	feature_id	No	Unique ID
path	linestring	No	Path in geographic world from start to end

Notes:

- The file will be present if there are any continuous tubes in the data.

2.6 Tube Segment

A *tube segment* models a section of a *tube* between two structures.

Field	Type	Optional	Comments
id	feature_id	No	Unique ID

in_conduit	feature_id	No	Previous conduit in run (if any)
out_conduit	feature_id	No	Next conduit in run (if any)
conduit_run	feature_id	No	Tube of which self is a part
housing	feature_id	No	Conduit, tube segment or route that is direct container
root_housing	feature_id	Yes	Route in which segment runs
in_structure	feature_id	Yes	Start structure
out_structure	feature_id	Yes	End structure
path	linestring	Yes	Path in geographic world from start to end
<feature-specific field>	...	Yes	Feature type-specific attribute
...	...	...	...

Notes:

- If *root_housing, in_structure, out_structure* and *path* are supplied their values must match those of *housing*
- *Feature-specific fields* will vary by tube type
- The package will contain a separate file for each conduit type present in the data.

2.7 Cable

A cable models a pyhsical cable within the network. Internally it consists of a chain of cable segments. Connections can be made at any structure along its path (not just at its end points).

Field	Type	Optional	Comments
id	feature_id	No	Unique ID
<tech>_count	integer	Yes	Number of fibers, copper pairs or coax wires. Tech is one of fiber, copper or coax
directed	boolean	No	True if direction is meaningful to user
path	linestring	No	Path in geographic world (from start to end)
placement_path	linestring	Yes	Path entered by user to set route (if routed manually)
offset_geom	linestring	Yes	Geoschematic path
<feature-specific field>	...	Yes	Feature type-specific attribute
...	...	...	...

Notes:

- *Feature-specific fields* will vary by cable type
- The package will contain a separate file for each cable type present in the data.

2.8 Cable Slack

A *cable slack* models a coil of cable within a structure.

Field	Type	Optional	Comments
id	feature_id	No	Unique ID
cable	feature_id	No	Cable of which self is a part
length	double	No	Length (in m)
housing	feature_id	No	Equipment or structure that is direct container
root_housing	feature_id	Yes	Structure in which self is housed

location	point	Yes	Location in geographic world
<feature-specific field>	...	Yes	Feature type-specific attribute
...	...		...

Notes:

- Every slack record must have a corresponding *cable segment* record. That segment must name the slack as its *housing* and have a length matching the slack record length
- If *location* is supplied it must match that of *housing*
- *Feature-specific fields* will vary by cable type
- The package will contain a separate file for each cable type present in the data.

2.9 Cable Segment

A *cable segment* models a section of a *cable* between two structures (or within a structure).

Field	Type	Optional	Comments
id	feature_id	No	Unique ID
cable	feature_id	No	Cable of which self is a part
directed	boolean	Yes	Must match cable.directed
length	double	Yes	Measured length (in m)
forward	boolean	No	True if self path runs in same direction as root_housing
in_segment	feature_id	No	Previous segment in chain (if any)
out_segment	feature_id	No	Next segment in chain (if any)
housing	feature_id	No	Conduit, tube segement, route, equipment or structure that is self's direct container
root_housing	feature_id	Yes	Route or structure in which self runs
in_structure	feature_id	Yes	Start structure
out_structure	feature_id	Yes	End structure
In_equipment	feature_id	Yes	Start equipment (if explicitly contained)
out_equipment	feature_id	Yes	End equipment (if explicitly contained)
path	linestring	Yes	Path in geographic world from start to end

Notes:

- If *root_housing*, *in_structure*, *out_structure* and *path* are supplied they must match those of *housing* (adjusted for *forward* flag, if necessary).
- For an *internal* segment, *in_structure*, *out_structure* and *root_housing* will have the same value.
- Fields *in_equipment* and *out_equipment* are only populated where containment cannot be inferred from fiber-level connectivity.

2.10 Connection

A *connection* models a fiber-level connection from one range of fibers (or ports) to another range of fibers (or ports).

Field	Type	Optional	Comments
id	feature_id	No	Unique ID
in_feature	feature_id	No	Cable segment or equipment connection originates at

in_side	string	No	Side of feature where connection sits ('in' or 'out')
in_low	integer	No	First fiber or port in range
in_high	integer	No	Last fiber or port in range
out_feature	feature_id	No	Cable segment or equipment runs to
out_side	string	No	Side of feature where connection sits ('in' or 'out')
out_low	integer	No	First fiber or port in range
out_high	integer	No	Last fiber or port in range
splice	boolean	No	True if in and out feature are both cable segments
housing	feature_id	No	Equipment or structure that is self's direct container
root_housing	feature_id	Yes	Structure in which self is located
location	point	Yes	Location of self's housing in geographic world

Notes:

- If *root_housing* and *location* are supplied they must match those of *housing*
- Direction of the connection is not important (has no meaning)

2.11 Line of Count

The line of count and line of count section features are used to model logical assignment of name and status to connected strands (fibers or pairs) in the network. The line of count feature holds the name and status of the assignment and an association to the originating feature for the assignment.

Field	Type	Optional	Comments
Id	feature_id	No	Unique ID
origin	feature_id	No	Cable segment or equipment line of count originates at. For equipment, this should be qualified with the side of the equipment the line of count originates at.
low_physical	integer	No	First strand assignment applies to.
high_physical	integer	No	Last strand assignment applies to.
low_logical	integer	No	Start of logical range
high_logical	integer	No	End of logical range
name	string	No	Assignment name
status	string	No	Assignment status
physical	boolean	No	Indicates that only status of assignment is to be shown.
route	linestring	Yes	Aggregation of line of count section geometry. Will be calculated if not present.

2.12 Line of Count Section

The line of count section feature relates line of count assignment to a cable segment or equipment ports. It describes how strands at the origin map to strands at the container, which along with the parent line of count feature, is used to calculate the assignment to strands at the container.

Field	Type	Optional	Comments
Id	feature_id	No	Unique ID

line_of_count	feature_id	No	Line of count feature this is a section for
container	feature_id	No	Cable segment or equipment the section applies to. For equipment, this should be qualified with the side of the equipment the section belongs to.
mapping	string	No	JSON string containing mapping information. See below for details.
route	linestring	Yes	Path of section. Will be calculated from container geometry if not present.

Notes:

- Each line of count section has a mapping field that describes how each strand at the origin maps to strands on the section's container. The mapping is a summary of the physical connectivity from the origin of the line of count to the container, and is used to calculate the actual assignment of strands at the container. For example, the mapping:

  ```
  {"1": [13], "2": [14,15], "3": [16], "4": [17] }
  ```

  Indicates that the strand 1 at the origin traces to the strand 13 at the container. Strand 2 at the origin traces to strands 14 and 15 at the container (since the trace passed though a splitter).

3 Field MetaData

A field metadata file provides information about each attribute of a feature type. Its structure is as follows:

Field	Type	Comments
name	string	Field name
type	string	Data type of field (see below)
unit	string	Unit in which value is represented e.g. 'mm'

The following data types are supported:

Data Type	DB Type	Comments
id	string	Feature ID
id_set	string	List of feature IDs, joined with ';'
string	string	String (optionally with max length)
integer	32-bit integer	Integer number
double	64-bit float	Floating point number
numeric	fixed point	Fixed point value (with precision and dp)
boolean	boolean	True/false
date	date	Date value
timestamp	timestamp	Date and time value
link	string	URL string
image	base64 string	Image as base64-encode string, with type prefix
file	base64 string	File as base64-encode string, with type prefix

Field metadata files are optional. Their main purpose is to permit unit conversion when loading data.

4 Package Metadata

The package metadata file provides global properties of the package. Its structure is:

Field	Type	Comments
property	string	Property name (see below)
value	<varies>	Property value (see below)

The following properties are supported:

Property	Value	Default	Comments
format	string	'cdif'	The package format
coord_system	integer	4326	EPSG code for the map projection in which coordinates are represented
boundary	EWKB string	-	Polygon geometry defining extent of data in package (in *coord_system*)

The package metadata file and all its properties are optional.

Copyright © 2021, IQGeo UK Ltd 2014 - 2023. All Rights Reserved. You may not reproduce this document in whole or in part without permission in writing from IQGeo at the following address:

IQGeo
Nine Hills Road
Cambridge, CB2 1GE
United Kingdom

WWW: https://www.iqgeo.com/

All contents of this document are subject to change without notice and do not represent a commitment on the part of IQGeo. Reasonable effort is made to ensure the accuracy of the information contained in the document. However, due to on-going product improvements and revisions, IQGeo and its subsidiaries do not warrant the accuracy of this information and cannot accept responsibility for errors or omissions that may be contained in this document.

Information in this document is provided in connection with IQGeo products. No license, express or implied to any intellectual property rights is granted by this document.

IQGeo encourages all users of its products to procure all necessary intellectual property licenses required to implement any concepts or applications and does not condone or encourage any intellectual property infringement and disclaims any responsibility related thereto. These intellectual property licenses may differ from country to country and it is the responsibility of those who develop the concepts or applications to be aware of and comply with different national license requirements.

Windows® is a registered trademark of Microsoft Corporation in the United States and/or other countries. The other names of actual companies and products mentioned herein are the trademarks of their respective owners.